

Orquestração da Modern Data Stack

De ETL para ELT com Snowflake e dbt no Apache Airflow

Disciplina de Extração de Dados

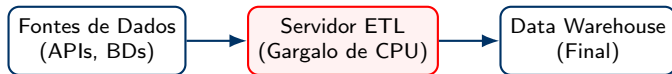
IBMEC

21 de maio de 2026

- Compreender a evolução do paradigma **ETL** para **ELT** e a **Modern Data Stack (MDS)**.
- Entender o papel central dos **Cloud Data Warehouses** (ex: Snowflake).
- Conhecer o **dbt (data build tool)**, a ferramenta padrão de mercado para transformação de dados.
- Visualizar como o **Apache Airflow** orquestra toda essa arquitetura complexa de ponta a ponta.

O Clássico: ETL (Extract, Transform, Load)

- Os dados são extraídos da fonte.
- São transformados em um servidor intermediário (caro, lento, gargalo).
- São carregados no Data Warehouse analítico final.



O Moderno: ELT (Extract, Load, Transform)

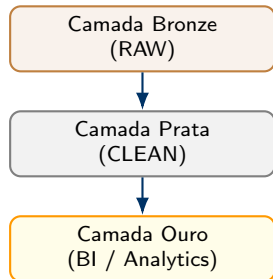
- Os dados são extraídos e jogados "crus" no Amazon S3 (Data Lake).
- São **carregados diretamente** no Data Warehouse (Snowflake) sem tratamento.
- São **transformados dentro do próprio DW**, usando o poder computacional massivo dele.



Arquitetura Medalhão (Medallion Architecture)

No paradigma ELT, como organizamos as transformações dentro do Data Warehouse? O mercado adotou a **Arquitetura Medalhão** (popularizada pela Databricks):

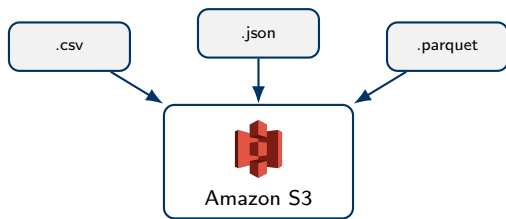
- **Bronze (RAW):** O dado *cru*, exatamente como veio da fonte. Sem tratamento, mas com histórico completo e confiável.
- **Prata (Silver / Staging):** O dado limpo, filtrado e padronizado (conversão de datas, tratamento nulos).
- **Ouro (Gold / Marts):** O dado agregado com a **regra de negócio** final, pronto para consumo em painéis e relatórios executivos.



O que é o Amazon S3?

O **Amazon S3** (Simple Storage Service) é o serviço de armazenamento de objetos da AWS.

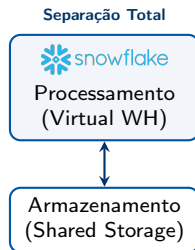
- **Data Lake:** Serve como o repositório central para armazenar dados brutos (Landing Zone).
- **Flexibilidade:** Suporta qualquer formato de arquivo (CSV, JSON, Parquet, Avro).
- **Escalabilidade:** Armazenamento praticamente infinito, altamente seguro e de baixíssimo custo.



O que é o Snowflake?

O **Snowflake** é um Cloud Data Warehouse (banco de dados analítico na nuvem) moderno.

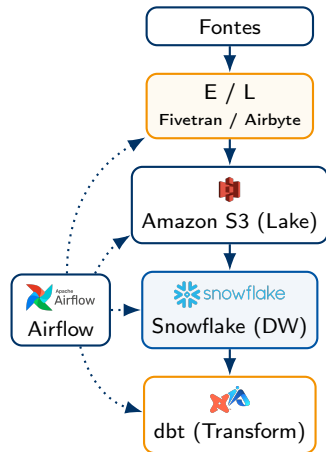
- **Separação de Storage e Compute:** Você paga pelo armazenamento e pelo poder computacional de forma independente.
- **Virtual Warehouses:** Clusters de computação elásticos que sobem e descem em segundos para processar consultas SQL.
- **Desempenho Massivo:** Otimizado para consultas analíticas sobre grandes volumes de dados.



O que é a Modern Data Stack (MDS)?

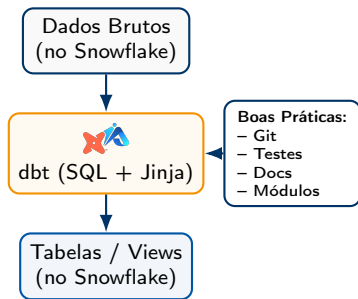
A *Modern Data Stack* é uma coleção de ferramentas em nuvem integradas, especializadas em cada etapa do pipeline:

- **E/L:** Fivetran, Airbyte (Ferramentas *No-Code* para carregar dados).
- **Storage:** Amazon S3 (Data Lake).
- **Compute:** Cloud DW (Snowflake, BigQuery). Separa compute de storage.
- **Transform (T):** dbt (data build tool).
- **Orquestrador:** Apache Airflow. Ele rege toda a orquestra.



O que é o dbt (data build tool)?

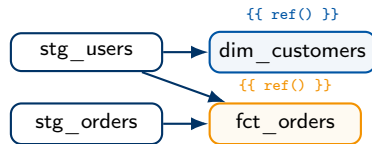
- O dbt é o "T" do padrão ELT.
- Ele **não extrai** nem **carrega** dados. Ele pressupõe que os dados **já estão no Snowflake**.
- **Propósito principal:** Trazer as melhores práticas de **Engenharia de Software** para analistas de dados que escrevem SQL.
- Permite criar *pipelines* de transformação escrevendo apenas comandos SELECT, e o dbt se encarrega de transformá-los em tabelas ou views reais no banco de dados.



Por que usar o dbt?

Se podemos transformar dados via Python/procedures, por que usar dbt?

- 1 **Modularidade:** SQL dividido em modelos pequenos e organizados (models).
- 2 **Linhagem (Jinja):** A função `{{ ref() }}` mapeia dependências automaticamente, gerando o Grafo de Linhagem.
- 3 **Testes Automatizados:** Regras de integridade (único, não-nulo) testadas em arquivos `.yaml`.
- 4 **Documentação:** Gera um site explicativo com relacionamentos de tabelas.



Grafo de Linhagem de Dados (DAG)

Estrutura de Pastas e Nomenclatura (dbt)

O dbt organiza o código SQL seguindo lógicas da **Arquitetura Medalhão** (Bronze, Prata, Ouro):

- **stg_ (Staging / Prata):** Modelos de "palco". Lêem a tabela bruta (Bronze). Fazem apenas limpeza, conversão de tipos (ex: texto para data) e padronização de nomes.
- **fct_ (Fact / Ouro):** Tabelas Fato. Trazem métricas de negócio (ex: total de amostras em conformidade).
- **dim_ (Dimension / Ouro):** Tabelas Dimensão. Trazem atributos descritivos (ex: cadastro de clientes, locais).

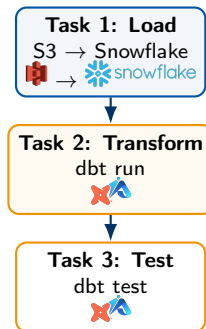
```
1 dbt_project_example/  
2 |-- dbt_project.yml  
3 |-- models/  
4 |   |-- staging/    (Camada Prata)  
5 |   |   |-- stg_qualidade_agua.sql  
6 |   |   |-- stg_clientes.sql  
7 |   |  
8 |   |-- marts/      (Camada Ouro)  
9 |   |   |-- fct_conformidade.sql  
10 |   |   |-- dim_pontos_coleta.sql  
11 |  
12 |-- tests/
```

Estrutura de Diretórios

Como o Airflow rege essa orquestra?

O **Apache Airflow** atua como o coordenador geral do fluxo ELT.

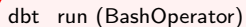
- 1 **Task 1 (Load)**: Airflow instrui o Snowflake a ler os arquivos do **Amazon S3** (COPY INTO).
- 2 **Task 2 (Transform)**: Airflow aciona o dbt para executar os modelos (dbt run).
- 3 **Task 3 (Test)**: Airflow aciona o dbt para testar a qualidade dos dados (dbt test).



O Problema da Caixa Preta e a solução "Cosmos"

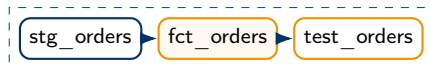
- **O Problema Clássico:** Rodar o dbt no Airflow via BashOperator deixa tudo dentro de **uma única "Task"**. Se falhar, não sabemos qual modelo falhou visualmente.
- **A Solução Moderna (Astronomer Cosmos):** Biblioteca que converte cada arquivo .sql do dbt em uma **Task individual** no grafo do Airflow!

Abordagem Clássica (Caixa Preta)



```
graph LR; A[dbt_run (BashOperator)]
```

Abordagem Cosmos



Implementação Prática: O Fluxo de Trabalho

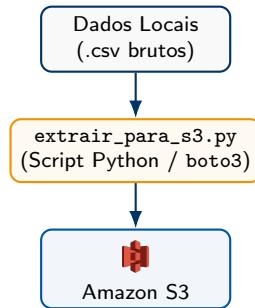
Para a demonstração prática da nossa aula de hoje, dividiremos a execução em 3 etapas claras:

- 1 **Extração & Ingestão (Python → S3):** Rodar o script Python localmente para subir os arquivos reais de qualidade da água.
- 2 **Carregamento (S3 → Snowflake):** Executar a primeira task da DAG do Airflow para rodar o comando `COPY INTO` no Snowflake.
- 3 **Transformação (Snowflake + dbt):** Deixar que o Cosmos no Airflow orquestre a execução do dbt para tratar os dados na camada Prata e Ouro.

Etapa 1: Extração e Carga no S3 com Python

Utilizamos a biblioteca **boto3** do Python para fazer a ingestão dos dados brutos locais diretamente para o S3:

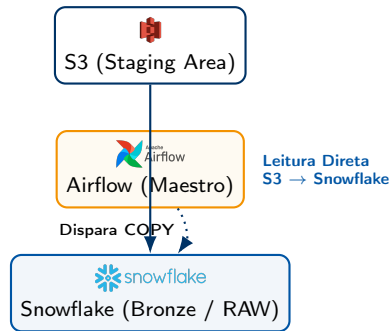
- **Origem:** Arquivos CSV brutos locais (ETA MCP.csv e ETA STN.csv) na pasta de dados brutos.
- **Script:** extrair_para_s3.py automatiza o upload.
- **Destino:** S3 Bucket larp-dados-aula19.
- **Tratamento:** Substituição de espaços por sublinhados nos arquivos enviados para o S3.



Etapa 2: Carregamento no Snowflake (Load)

Na segunda etapa, o **Airflow** orquestra o carregamento dos arquivos brutos do S3 para a tabela no **Snowflake** usando o poder do DW:

- **Tarefa do Airflow:**
SQLExecuteQueryOperator se conecta ao Snowflake.
- **Nativo e Rápido:** Comando COPY INTO transfere os dados diretamente da nuvem para o DW.
- **Tratamento de Arquivo:** Configurado com delimitador ';' e codificação iso-8859-1 (Latin1) para ler a qualidade da água corretamente.



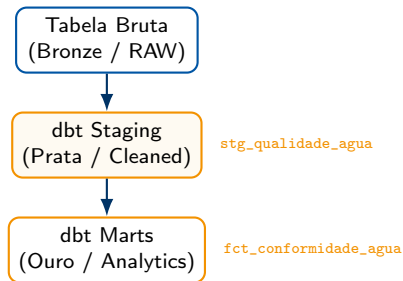
Etapa 2 (Código): Tarefa de Carregamento (Airflow)

```
1 # Tarefa que orquestra a carga do S3 para o Snowflake
2 load_s3_to_snowflake = SQLExecuteQueryOperator(
3     task_id='carregar_dados_s3_para_snowflake',
4     conn_id='snowflake_default',
5     sql="""
6     COPY INTO RAW.PUBLIC.QUALIDADE_AGUA_BRUTA
7     FROM 's3://larp-dados-aula19/dados_brutos/'
8     CREDENTIALS = (AWS_KEY_ID='***' AWS_SECRET_KEY='***')
9     FILE_FORMAT = (
10         TYPE = CSV
11         FIELD_DELIMITER = ';'
12         SKIP_HEADER = 1
13         ENCODING = 'iso-8859-1'
14     )
15     PATTERN = '.*\\.csv';
16 """)
17 )
18
```

Etapa 3: Transformação e Refinamento com dbt (Transform)

Com os dados brutos no Snowflake (Bronze), o **dbt** entra em ação para transformar e refinar os dados:

- **Camada Prata (Staging):** Limpeza de tipos, datas, strings e padronização de colunas (`stg_qualidade_agua.sql`).
- **Camada Ouro (Marts):** Agregações analíticas e cálculo das taxas de conformidade da água (`fct_conformidade_agua.sql`).
- **Orquestração Cosmos:** O Cosmos converte esses modelos em tarefas Airflow individuais e monitoráveis.



Etapa 3 (Código): Entendendo o Staging (Prata)

```
1 with source as (  
2     -- {{ source() }}: Le da tabela bruta  
3     select * from {{ source('raw_data', 'qualidade_agua_bruta') }}  
4 ),  
5 renamed as (  
6     select  
7         n_amostra::int as id_amostra,  
8         to_date(execucao_coleta, 'DD/MM/YYYY') as data_coleta,  
9         ponto_amstral, parametro, tipo_amostra,  
10        saa as sistema_abastecimento,  
11        -- try_cast/replace: arruma virgula de decimal  
12        try_cast(replace(replace(resultado, '.', ''), ',', '.')) as  
13        float) as valor_resultado,  
14        -- case when: traduz Sim/Nao para True/False  
15        case when em_conformidade = 'Sim' then true else false end  
16        as flg_em_conformidade  
17    from source  
18 )  
19 select * from renamed
```

O que está acontecendo?

- {{ source() }}: Cria o vínculo com os dados originais no banco.
- ::int / to_date(): **Conversão (Cast)**. Transforma campos de texto em inteiros ou datas válidas.
- replace(): **Limpeza**. Converte o padrão brasileiro numérico (1.000,50) para o padrão do banco de dados (1000.50).
- case when: **Padronização**. Troca textos por booleanos (True/False), melhorando a performance analítica.

Etapa 3 (Código): Fato e Regra de Negócio (Ouro)

```
1 with stg_qualidade as (  
2     -- {{ ref() }}: Le da camada prata (Linhagem)  
3     select * from {{ ref('stg_qualidade_agua') }}  
4 )  
5 select  
6     ponto_amostrais,  
7     parametro,  
8     count(id amostra) as total amostras,  
9  
10    count_if(flge conformidade) as amostras em conformidade,  
11    count_if(not flge conformidade) as amostras fora conformidade,  
12  
13    -- Calculo da regra de negocio (%)  
14    round(100.0 * count_if(flge conformidade) / count(id amostra),  
15          2) as taxa conformidade_pct  
16 from stg_qualidade  
17 group by ponto_amostrais, parametro
```

O que está acontecendo?

- **{{ ref() }}**: **Mágica da Linhagem!** O dbt sabe que tem que rodar o stg_ antes dessa fct_.
- **count_if()**: Conta quantas linhas passaram no teste toxicológico (pois criamos o True/False na etapa anterior).
- **Cálculo de Porcentagem (%)**: A **regra de negócio** final calculando a taxa de aprovação da água.
- **group by**: Agrupa detalhes transformando em **resumo executivo** por local.

Etapa 3: Resultado Analítico (A Tabela Fato)

Visão Final: fct_conformidade_agua

Ponto Amostral	Parâmetro	Total Amostras	Conformes	Não Conformes	Taxa Conformidade (%)
ETA MCP - Saída	Cloro Residual	150	148	2	98.67
ETA MCP - Saída	Turbidez	150	150	0	100.00
ETA STN - Reservatório	Coliformes Totais	80	75	5	93.75
ETA STN - Reservatório	Cor Aparente	80	79	1	98.75

Como ler essa tabela (Insights de Negócio):

- Reduzimos milhares de linhas brutas em um **resumo executivo** limpo e padronizado.
- O diretor de qualidade bate o olho e vê que a **Taxa de Conformidade** de Coliformes Totais no ponto *ETA STN* está mais baixa (93.75%), permitindo ação rápida.
- Essa é a tabela Ouro, perfeita para ser conectada a um **Dashboard (Power BI, Tableau)**, fechando com chave de ouro o nosso pipeline!

Conclusão: A visão de mercado

Hoje, saber orquestrar a tríade (**Cloud DW + dbt + Airflow**) é um dos conhecimentos mais requisitados e bem pagos para Engenheiros de Dados de todo o mundo. O foco não é apenas programar algoritmos, mas orquestrar o movimento da informação de forma confiável.